

2.3 语音唤醒

本节课学习使通过程序用语音大模型, 说出预先设定好的唤醒词, 对语音设备进行唤醒。

注意：如果是使用我们的机器人镜像，在开启本玩法前需要参考“02 多模态大模型应用/2.2 多模态大模型之语音控制/3. 玩法开启及关闭步骤”，将玩法自启程序关闭。

体验玩法结束后需要参考上述路径重新将自启打开。

1. WonderEchoPro 唤醒

1.1. 设备检查

我们需要查询我们接入的 WonderEchoPro/环形麦克风（后面统称为**语音设备**）的 USB 设备号名字，具体的步骤如下（**注意：**查询的过程不要接入其他的 USB 设备）。

1) 首先**断开**语音设备，然后在命令行终端输入以下指令。

```
ls /dev | grep USB
```

```
> ls /dev | grep USB  
ttyCH341USB0
```

2) 接着我们将语音设备**接入**到我们的主板的 USB 接口上，然后输入指令，查看当前主板的 USB 设备名字。

```
ls /dev | grep USB
```

```
> ls /dev | grep USB
```

3) 可以看到接入语音设备之后，出现了一个 USB 端口为 **ttyCH341USB1**（需要记住这个设备名字），不同的主控输出名字可能会有不同。

```
> ls /dev | grep USB  
ttyCH341USB0  
ttyCH341USB1
```

1.2. 唤醒测试

- 1) 新建一个命令行终端，然后输入指令，编辑 wakeup_demo.py

```
vim ~/large_models/wakeup_demo.py
```

- 2) 按下键盘的按键 i 进入到编辑模式，端口 port 修改为查询到的端口编号。

```
9 port = '/dev/ttyCH341USB1'  
10 kws = awake.WonderEchoPro(port)
```

- 3) 修改完后按“ESC”键退出编辑，shift+ “:”，输入 wq 完成保存退出。

```
COMMAND → wakeup_demo.py[+]  
:wq
```

- 4) 接着来到系统的界面当中，运行 wakeup_demo 的程序，对着 AI 语音盒 (WonderEchoPro) 说出：“hello hiwonder”，会输出以下内容,如果输出了 hello hiwonder，说明固件烧录成功，提示词可以正常使用。

```
python3 ~/large_models/wakeup_demo.py
```

```
www.hiwonder.com  
sh: 1: pinctrl: not found  
start...  
hello hiwonder
```

2. 六路环形麦克风

同样的，我们可以通过 type-c（接环形麦克风）转 USB 线（连接树莓派或者 Jetson 主控的 USB 端口）。

2.1. 设备检查

在 Jetson 的主控上，使用 Nomachine 远程连接工具连接到主控，来到系统的界



面当中，看到系统桌面左侧出现了六路环形麦克风标志说明已经正常接入。

2.2. 唤醒测试

- 1) 新建一个命令行终端，然后输入指令，编辑 `wakeup_demo.py`。

```
vim ~/large_models/wakeup_demo.py
```

```
> vim ~/large_models/wakeup_demo.py
```

- 2) 按下键盘的按键 `i` 进入到编辑模式，端口 `port` 修改为查询到的端口编号。
将使用 `WonderEchoPro` 注释（前面加上 `#`号），然后使用 11 行的环形麦克风设备作为音频输入，如下图红框的内容。

```
9 port = '/dev/ttyCH341USB1'  
10 #kws = awake.WonderEchoPro(port)  
11 kws = awake.CircleMic(port)  
12
```

- 3) 修改完后按“`ESC`”键退出编辑，`shift+ “:”`，输入 `wq` 完成保存退出。

```
COMMAND wake_demo.py  
:wq
```

- 4) 接着在命令行终端里面输入指令。

```
python3 ~/large_models/wakeup_demo.py
```

```
> python3 ~/large_models/wakeup_demo.py
```

- 5) 等待 30s 左右初始化设置之后，说出唤醒词 “`hello hiwonder`”。

```
www.hiwonder.com  
sh: 1: pinctrl: not found  
start...  
hello hiwonder
```

3. 程序简要分析

这是一个基于 Python 的唤醒词检测脚本，使用 `speech` 模块处理音频输入并检测特定唤醒词（如“`hello hiwonder`”）

程序源码路径：/home/ubuntu/large_models/wakeup_demo.py

```
1 #!/usr/bin/env python3
2 # encoding: utf-8
3 # @Author: Aiden
4 # @Date: 2024/11/27
5 import os
6 import time
7 from speech import awake
8
9 port = '/dev/ttyCH341USB1'
10 kws = awake.WonderEchoPro(port)
11 # kws = awake.CircleMic(port)
12
13 try: # 如果有风扇，检测前推荐关掉减少干扰
14     os.system('pinctrl FAN_PWM op dh')
15 except:
16     pass
17
18 kws.start() # 开始检测
19 print('start...')
20 while True:
21     try:
22         if kws.wakeup(): # 检测到唤醒
23             print('hello hiwonder')
24             time.sleep(0.02)
25     except KeyboardInterrupt:
26         kws.exit() # 关闭处理
27         try:
28             os.system('pinctrl FAN_PWM a0')
29         except:
30             pass
31         break
32
```

3.1. 导入必要模块

```
5 import os
6 import time
7 from speech import awake
```

os：模块用于处理文件路径。

time：提供时间延迟功能，避免检测频率过高。

speech：核心模块，用于处理音频输入和唤醒词检测。

3.2. 初始化 wonderecho 类

```
9 port = '/dev/ttyCH341USB1'
10 kws = awake.WonderEchoPro(port)
11 # kws = awake.CircleMic(port)
```

3.3. 尝试关闭树莓派 5 散热器

```
13 try: # 如果有风扇，检测前推荐关掉减少干扰
14     os.system('pinctrl FAN_PWM op dh')
15 except:
16     pass
```

功能：尝试关闭风扇（通过执行系统命令 `pinctrl FAN_PWM op dh`），以减少风扇噪声对唤醒词检测的影响。

异常处理：如果命令无法执行（如硬件不支持），程序继续运行而不会报错。

3.4. 唤醒词检测主循环

```
18 kws.start() # 开始检测
19 print('start...')
```

调用 `kws.start()` 开启唤醒词检测线程。

输出 `start...`，提示检测已启动。

3.5. 主程序逻辑

```
20 while True:
21     try:
22         if kws.wakeup(): # 检测到唤醒
23             print('hello hiwonder')
24             time.sleep(0.02)
25     except KeyboardInterrupt:
26         kws.exit() # 关闭处理
27         try:
28             os.system('pinctrl FAN_PWM a0')
29         except:
30             pass
31         break
32
```

每次调用时，检测是否识别到唤醒词。如果检测到唤醒词，输出 `keyword detected`。

通过 `time.sleep(0.02)` 限制检测频率

按下 `Ctrl+C`（触发 `KeyboardInterrupt`）时，退出检测循环。

调用 `kws.exit()` 停止唤醒词检测。

恢复风扇状态。

4. 功能延伸

4.1. 唤醒提示词修改

下面我们学习修改唤醒后显示的字符。

1) 我们以唤醒 “`hello hiwonder`” 然后打印输出 “`hello`” 为例，在

large_models 目录下输入指令：

```
vim wakeup_demo.py
```

```
vim wakeup_demo.py
```

2) 点击键盘“i”，下方会显示 INSERT，然后找到 `print('hello hiwonder')` 修改成 `print('hello')`

```
i
```

```
21     try:
22         if kws.wakeup(): # 检测到唤醒
23             print('hello')
24             time.sleep(0.02)
25     except KeyboardInterrupt:
26         kws.exit() # 关闭处理
27     try:
28         os.system('pinctrl FAN_PWM a0')
29     except:
30         pass
31     break
32
```

3) 修改完后按“ESC”键退出编辑，shift+“:”，输入 wq 完成保存退出。

```
wq
```

```
COMMAND → wakeup_demo.py[+]
:wq
```

4) 然后输入以下指令：

```
python3 wakeup_demo.py
```

```
start...
hello
```

4.2. WonderEchoPro 固件制作

如果我们想制作更加更加丰富的唤醒词或者别的语音口令内容，可以参考“附录 \1.烧录固件工具\WonderEchoPro 固件制作”文档。